

Taller — Pokédex de Consola

Introducción a APIs con JavaScript · Guía para el estudiante

En este taller vas a construir una **Pokédex funcional** en JavaScript que consulta datos reales desde internet, usando exactamente el mismo patrón que ya viste en clase: `fetch` → `await` → verificar `response.ok` → `.json()` → navegar el objeto.

Al terminar vas a tener un mini sistema con funciones reutilizables que:

- Busca cualquier Pokémon por nombre
- Muestra su ficha completa (tipo, stats, habilidades)
- Compara dos Pokémon por una stat específica
- Encuentra el Pokémon más fuerte de una lista

Este es el mismo tipo de código que se usa en apps reales para consumir APIs de productos, usuarios o servicios externos.

🕒 Plan del taller — 60 minutos

Parte	Qué vas a construir	Tiempo
Parte 1	Conexión y exploración de la respuesta	10 min
Parte 2	Función <code>buscarPokemon()</code>	10 min
Parte 3	Función <code>mostrarFicha()</code>	10 min
Parte 4	Función <code>compararPokemon()</code>	15 min
Parte 5	Función <code>pokemonMasFuerte()</code>	15 min

📁 Antes de empezar

Vas a necesitar `prompt-sync` si quieres pedirle datos al usuario por teclado (igual que en `moneda.js`). En la terminal, dentro de tu proyecto:

```
npm install prompt-sync
```

La API que usaremos es **pública y gratuita** — no necesita registro ni llave: `https://pokeapi.co`

Recuerda: `fetch` ya viene incluido en Node.js (18 o superior). No necesitas instalar nada para hacer peticiones.

Parte 1 — Conexión y exploración de la respuesta 10 min

Antes de construir funciones, necesitas entender **qué forma tiene** lo que devuelve la API. El endpoint base es:

```
https://pokeapi.co/api/v2/pokemon/{nombre}
```

Reemplaza `{nombre}` con cualquier nombre **en minúsculas**: `pikachu`, `charizard`, `bulbasaur`.

⇒ Paso 1

Crea un archivo `explorar.js`. Dentro de una función `async`, haz un `fetch` a la URL de arriba con el pokémon que quieras, espera la respuesta con `await`, imprime el `status` de la respuesta, y luego traduce el resultado con `.json()` (con su propio `await`). Guarda ese resultado en una variable llamada `datos`.

⇒ Paso 2

Imprime `datos` completo con `console.log` y revisa su estructura. Vas a encontrar, entre otras, estas claves — son las que vamos a usar en todo el taller:

Clave	Tipo	Qué contiene
<code>name</code>	texto	Nombre del Pokémon
<code>id</code>	número	Número en la Pokédex
<code>height</code>	número	Altura (en decímetros)
<code>weight</code>	número	Peso (en hectogramos)
<code>types</code>	array de objetos	Tipos del Pokémon
<code>stats</code>	array de objetos	Stats base (hp, attack, defense...)
<code>abilities</code>	array de objetos	Habilidades

🔗 **Forma real de un elemento de `types` :**

```
{ slot: 1, type: { name: "electric", url: "..."} }
```

Para llegar solo al nombre del tipo: `datos.types[0].type.name`

🔗 **Forma real de un elemento de `stats` :**

```
{ base_stat: 35, stat: { name: "hp"} }
```

⇒ Ejercicio 1

Usando la variable `datos` que ya tienes, imprime en consola:

1. Todos los tipos del pokémon (puede tener más de uno)
2. Todas sus stats, con su nombre y su valor
3. Todas sus habilidades

Usa un bucle (`for` o `for...of`) para recorrer cada lista. No escribas `datos.types[0]` a mano — recórrelas todas, sin importar cuántos elementos tengan.

🔗 Pistas

- Para `types` : recorre `datos.types` e imprime `t.type.name` en cada vuelta.
- Para `stats` : recorre `datos.stats` e imprime `s.stat.name` y `s.base_stat` .
- Para `abilities` : recorre `datos.abilities` e imprime `a.ability.name` .

Parte 2 — Función `buscarPokemon()` 10 min

Ahora vas a encapsular la consulta en una función reutilizable.

¿Por qué una función? Porque en un sistema real no repetimos `fetch(url)` cada vez que necesitamos un dato. Lo ponemos en una función, y desde cualquier parte del programa la llamamos con un nombre. Si la URL de la API cambia algún día, solo cambias un lugar.

La función debe:

- Recibir el **nombre** de un pokémon como parámetro.
- Construir la URL usando ese nombre **en minúsculas** (recuerda: la API no acepta mayúsculas).
- Hacer la petición con `fetch` y `await` .
- Verificar `respuesta.ok` **antes** de traducir el resultado.
- Si la respuesta no salió bien: mostrar un mensaje con el `status` , y retornar `null` .
- Si salió bien: retornar el objeto ya traducido con `.json()` .

🔗 Esqueleto para guiarte (sin completar):

```
async function buscarPokemon(nombre) {  
  // 1. Construye la URL con el nombre en minúsculas (toLowerCase())  
  // 2. await fetch(...)  
  // 3. if (!respuesta.ok) { avisa el status y retorna null }  
  // 4. si todo salió bien, retorna await respuesta.json()  
}
```

⇒ Ejercicio 2

Crea `pokedex.js` con tu función `buscarPokemon()` . Pruébala buscando **tres pokémon distintos** (los que quieras). Para cada uno, imprime su `name` y su `id` — pero primero verifica que el resultado **no sea** `null` antes de intentar leer esos datos. Prueba también con un nombre que no exista y confirma que tu función avisa el error en vez de romperse.

🔗 Recuerda

`buscarPokemon` es `async` , así que para usar su resultado necesitas `await` también donde la llares — y esa llamada debe estar, a su vez, dentro de otra función `async` (o usarse con `.then()`).

Parte 3 — Función `mostrarFicha()` 10 min

Ya tienes una función que **consigue** los datos. Ahora necesitas una que los **muestre** de forma legible.

Esta separación es intencional: `buscarPokemon()` se encarga de *traer* el dato. `mostrarFicha()` se encarga de *mostrarlo*. Si mañana quisieras guardar la ficha en un archivo en vez de imprimirla, solo tocas esta función — la otra queda intacta.

La función `mostrarFicha(datos)` debe:

- Recibir el objeto de un pokémon como parámetro.
- Si `datos` es `null` , avisar que no hay nada que mostrar y salir de la función con `return` .
- Mostrar el nombre (en mayúscula) y el número de Pokédex.
- Mostrar todos los tipos, separados por " / " .
- Convertir y mostrar la altura en centímetros (`height * 10`) y el peso en kilogramos (`weight / 10`).

- Recorrer y mostrar **todas** las stats, con su nombre y su valor.
- Recorrer y mostrar **todas** las habilidades. Si una habilidad tiene `is_hidden` en verdadero, indícalo como "(oculta)".

 Para unir una lista de textos con " / ":

Primero arma un array solo con los nombres de los tipos (recorriendo `datos.types`), y luego usa el método de array `.join(" / ")` sobre ese array.

 Esqueleto para guiarte (sin completar):

```
function mostrarFicha(datos) {
  // 1. if (!datos) { avisa y return }
  // 2. imprime nombre en mayúscula + id
  // 3. arma un array de nombres de tipos y únelo con join(" / ")
  // 4. calcula altura en cm y peso en kg, imprímelos
  // 5. recorre datos.stats e imprime cada nombre + base_stat
  // 6. recorre datos.abilities e imprime cada nombre (+ "(oculta)" si aplica)
}
```

⇒ **Ejercicio 3**

Busca (con `buscarPokemon()`) y muestra la ficha completa (con `mostrarFicha()`) de **dos pokémon** que no hayas usado todavía.

 Para responder por escrito

¿Por qué es útil que `buscarPokemon()` y `mostrarFicha()` sean funciones separadas, en vez de un solo bloque de código que haga todo junto?

Parte 4 — Función `compararPokemon()` 15 min

Esta función recibe **dos nombres y una stat**, consulta ambos pokémon, y dice cuál tiene el valor más alto en esa stat.

Las stats disponibles son: `hp`, `attack`, `defense`, `special-attack`, `special-defense`, `speed`.

Como la stat viene como texto pero el valor está adentro de un array de objetos, primero vas a construir una función auxiliar más pequeña.

4.1 — Función auxiliar `obtenerStat(datos, nombreStat)`

Debe:

- Recorrer `datos.stats` buscando el elemento cuyo `stat.name` sea igual a `nombreStat`.
- Si lo encuentra, retornar su `base_stat`.
- Si termina de recorrer todo y no lo encontró, retornar `null`.

 Esqueleto:

```
function obtenerStat(datos, nombreStat) {
  // recorre datos.stats con un for
  // si stat.name === nombreStat, retorna base_stat (usa "return" para cortar ahí mismo)
  // si el for termina sin encontrar nada, retorna null
}
```

4.2 — Función `compararPokemon(nombre1, nombre2, stat)`

Debe:

- Buscar ambos pokémon con `buscarPokemon()` (recuerda el `await` en cada uno).
- Si alguno vino `null`, avisar que no se puede comparar y salir con `return`.
- Obtener el valor de la stat pedida para cada uno, con `obtenerStat()`.
- Si la stat no existe en ninguno (viene `null`), avisar cuáles son las stats válidas y salir.
- Comparar los dos valores e imprimir quién gana (o si hay empate).

⇒ **Ejercicio 4**

1. Compara `snorlax` vs `machop` en la stat que consideres más relevante para ellos. Deja un comentario justificando tu elección.
2. Compara dos pokémon de tu elección en `defense`.
3. Intenta comparar usando una stat que **no exista** (por ejemplo `"fuerza"`). ¿Qué debería imprimir tu función? Verifica que efectivamente lo haga.

Parte 5 — Función `pokemonMasFuerte()` 15 min

Esta es la función más completa del taller. Recibe una **lista de nombres** y una **stat**, consulta todos, y retorna el nombre del pokémon con el valor más alto en

esa stat.

Esta función integra todo lo que ya construiste: usa `buscarPokemon()` para cada nombre, usa `obtenerStat()` para extraer el valor, recorre la lista buscando el máximo, y retorna el ganador.

La función `pokemonMasFuerte(listaNombres, stat)` debe:

- Guardar en dos variables el mejor nombre encontrado hasta ahora y su mejor valor (empieza el valor en algo imposible de superar por abajo, como `-1`).
- Recorrer `listaNombres`. Para cada nombre: buscarlo con `buscarPokemon()`.
- Si vino `null`, saltarlo y seguir con el siguiente (`continue`).
- Obtener su valor de la stat con `obtenerStat()`. Si es `null`, saltarlo también.
- Si su valor es mayor al mejor guardado hasta ahora, actualizar ambas variables.
- Al terminar de recorrer toda la lista, mostrar quién ganó y retornar su nombre.

 **Sobre recorrer con `await` adentro de un `for`**

Sí se puede usar `await` dentro de un `for` normal, siempre que el `for` esté dentro de una función `async`. Cada vuelta del bucle espera su propia respuesta antes de seguir con la siguiente — igual que hiciste en la guerra de pokémones de la clase.

⇒ **Ejercicio 5 — Desafío final**

1. Arma tu propio equipo de **6 pokémon** (los que quieras) en un array.
2. Usa `pokemonMasFuerte()` para encontrar cuál es el más fuerte en `attack`.
3. Encuentra cuál es el más fuerte en `defense`.
4. Muestra la ficha completa del pokémon ganador en `attack`, usando `mostrarFicha()`.

Restricción: todo debe hacerse usando las funciones que ya construiste. No hagas `fetch` directamente en este ejercicio — pasa siempre por `buscarPokemon()`.

🌐 ¿Dónde aparece esto en la vida real?

Lo que construiste hoy no es solo un juego. Es exactamente el patrón que usan los sistemas reales:

Lo que hiciste aquí	Equivalente en producción
<code>buscarPokemon(nombre)</code>	<code>buscarProducto(id)</code> , <code>obtenerUsuario(email)</code>
<code>mostrarFicha(datos)</code>	<code>renderizarTarjeta(producto)</code> , <code>imprimirPerfil(usuario)</code>
<code>compararPokemon()</code>	Comparador de precios, comparador de candidatos en un ATS
<code>pokemonMasFuerte()</code>	Algoritmo de ranking, recomendación del mejor plan, producto más vendido
Verificar <code>respuesta.ok</code> antes de procesar	Manejo de errores en cualquier integración con API externa

La estructura de **funciones con responsabilidad única** que usaste hoy es uno de los principios más importantes del desarrollo real. Cada función hace una sola cosa — y eso las hace reutilizables, fáciles de probar y fáciles de modificar.

API usada: PokéAPI — `https://pokeapi.co` (gratuita, sin autenticación, documentación pública)